

OS_project02_10831

2022083409 컴퓨터소프트웨어학부 김승관

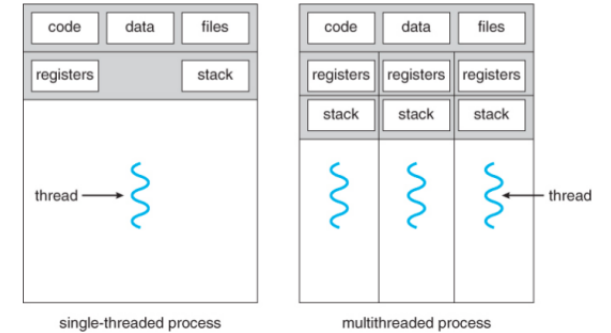
Design

- kernel/proc.c
- pagetable_t proc_pagetable(struct proc *p)
 - kernel/proc.c static struct proc* allocproc(void)에서 process의 initial pagetable을 생성하는 proc_pagetable()을 호출한다.
 - proc_pagetable()에서는 pagetable을 새로 만들고 trampoline을 TRAMPOLINE (MAXVA - PGSIZE)에 PGSIZE만큼 mapping 해준다. 이후에 process의 trapframe을 TRAPFRAME (TRAMPOLINE - PGSIZE)에 PGSIZE만큼 mapping 해준다.
 - Thread는 main thread와 pagetable은 공유하지만, register state를 저장하고 있는 trapframe은 공유해서는 안된다. 따라서 thread를 생성할 때 trapframe이 같은 위치에 mapping되지 않도록 수정해야 한다.

allocproc()

```
// An empty user page table.
p->pagetable = proc_pagetable(p);
if(p->pagetable == 0){
    freeproc(p);
    release(&p->lock);
    return 0;
}
```

multithreaded process



proc_pagetable()

```
pagetable_t
proc_pagetable(struct proc *p)
{
    pagetable_t pagetable;

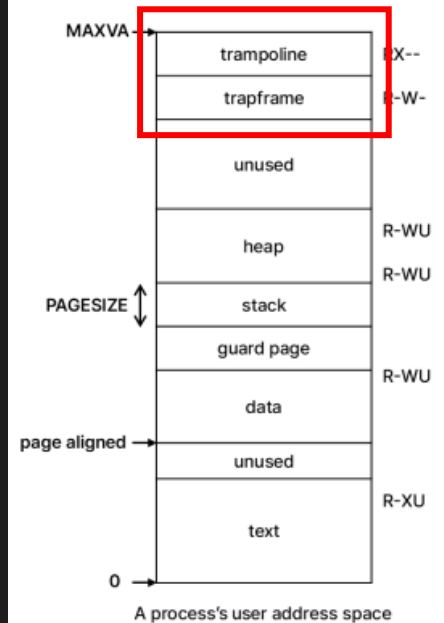
    // An empty page table.
    pagetable = uvmcreate();
    if(pagetable == 0)
        return 0;

    // map the trampoline code (for system call return)
    // at the highest user virtual address.
    // only the supervisor uses it, on the way
    // to/from user space, so not PTE_U.
    if(mappages(pagetable, TRAMPOLINE, PGSIZE,
        (uint64)trampoline, PTE_R | PTE_X) < 0){
        uvmfree(pagetable, 0);
        return 0;
    }

    // map the trapframe page just below the trampoline page, for
    // trampoline.S.
    if(mappages(pagetable, p->trapframe_va = TRAPFRAME, PGSIZE,
        (uint64)(p->trapframe), PTE_R | PTE_W) < 0){
        uvmunmap(pagetable, TRAMPOLINE, 1, 0);
        uvmfree(pagetable, 0);
        return 0;
    }

    return pagetable;
}
```

user address space



Design

- kernel/proc.c int fork(void)
 - 부모 process를 복사해 자식 process를 생성한다.
 - 자식 process는 부모 process의 pagetable을 uvmcopy로 복사한다. = 연산자로 값을 전달하면 공유하는 것이기에 uvmcopy를 사용해야 한다.
 - trapframe도 참조 연산자 *를 사용해 부모 process의 trapframe 값을 복사한다.
 - Thread의 경우 main thread와 pagetable을 공유하기 때문에 uvmcopy로 복사하는 것이 아닌, = 연산자를 사용해 main thread의 pagetable을 공유할 수 있게 해야한다.

fork()

```
int
fork(void)
{
    int i, pid;
    struct proc *np;
    struct proc *p = myproc();

    // Allocate process.
    if((np = allocproc()) == 0){
        return -1;
    }

    // Copy user memory from parent to child.
    if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){
        freeproc(np);
        release(&np->lock);
        return -1;
    }
    np->sz = p->sz;

    // copy saved user registers.
    *(np->trapframe) = *(p->trapframe);

    // Cause fork to return 0 in the child.
    np->trapframe->a0 = 0;

    // increment reference counts on open file descriptors.
    for(i = 0; i < NOFILE; i++)
        if(p->ofile[i])
            np->ofile[i] = filedup(p->ofile[i]);
    np->cwd = idup(p->cwd);

    safestrcpy(np->name, p->name, sizeof(p->name));

    pid = np->pid;

    release(&np->lock);

    acquire(&wait_lock);
    np->parent = p;
    release(&wait_lock);

    acquire(&np->lock);
    np->state = RUNNABLE;
    release(&np->lock);

    return pid;
}
```

Design

- kernel/exec.c
- int exec(char *path, char **argv)
 - 부모 process를 복사해 child process를 생성하는 fork()와 달리 exec()은 현재 process를 새로운 process로 바꾸는 함수이다.
 - proc_pagetable()로 pagetable을 새로 만든 뒤 그 안에 새로운 process 실행을 위한 정보들을 할당하거나 불러온다.
 - 이후 기존 process의 pagetable과 교체한다.
 - thread가 exec()을 실행할 때 pagetable을 교체하는 과정에서 회수되지 않은 trapframe이 없도록 신경 써야 한다.

exec()

```
if((pagetable = proc_pagetable(p)) == 0)
    goto bad;

// Load program into memory.
for(i=0, off=elf.phoff; i<elf.phnum; i++, off+=sizeof(ph)){
    if(readi(ip, 0, (uint64*)&ph, off, sizeof(ph)) != sizeof(ph))
        goto bad;
    if(ph.type != ELF_PROG_LOAD)
        continue;
    if(ph.memsz < ph.filesz)
        goto bad;
    if(ph.vaddr + ph.memsz < ph.vaddr)
        goto bad;
    if(ph.vaddr % PGSIZE != 0)
        goto bad;
    uint64 sz1;
    if((sz1 = uvmalloc(pagetable, sz, ph.vaddr + ph.memsz, flags2perm(ph.flags))) == 0)
        goto bad;
    sz = sz1;
    if(loadseg(pagetable, ph.vaddr, ip, ph.off, ph.filesz) < 0)
        goto bad;
}
iunlockput(ip);
end_op();
ip = 0;

// Commit to the user image.
oldpagetable = p->pagetable;
p->pagetable = pagetable;
p->sz = sz;
p->trapframe->epc = elf.entry; // initial program counter = main
p->trapframe->sp = sp; // initial stack pointer
proc_freepagetable(oldpagetable, oldsz);
return argc; // this ends up in a0, the first argument to main(argc, argv)
```

Design

- kernel/proc.c int growporc(int n)
 - sbrk syscall을 보면 growproc()을 사용해서 process의 heap 크기를 동적으로 조절한다.
 - thread는 main thread와 heap 영역을 공유하기 때문에 이 점을 유의해서 수정해야 한다.

sysproc.c

```
uint64
sys_sbrk(void)
{
    uint64 addr;
    int n;

    argint(0, &n);
    addr = myproc()->sz;
    if(growproc(n) < 0)
        return -1;
    return addr;
}
```

growproc()

```
int
growproc(int n)
{
    uint64 sz;
    struct proc *p = myproc();
    sz = p->sz;
    if(n > 0){
        if((sz = uvmmalloc(p->pagetable, sz, sz + n, PTE_W)) == 0) {
            return -1;
        }
    } else if(n < 0){
        sz = uvmmdealloc(p->pagetable, sz, sz + n);
    }
    p->sz = sz;
    return 0;
}
```

Design

- kernel/proc.c int kill(int pid)
 - 특정 pid를 지닌 process의 killed를 1로 만든다. process state가 SLEEPING일 경우 RUNNABLE로 바뀌서 scheduler가 확인할 수 있게 한다.
 - 이후 usertrap에서 exit(-1)로 해당 process를 종료한다.
 - thread의 경우 pid를 main thread와 공유하기 때문에, 특정 pid를 찾자마자 return으로 종료하는 것이 아니라 proc table을 다 돌면서 특정 pid를 갖는 process의 killed를 전부 바꿔줘야 한다.

```
int
kill(int pid)
{
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->pid == pid){
            p->killed = 1;
            if(p->state == SLEEPING){
                // Wake process from sleep().
                p->state = RUNNABLE;
            }
            release(&p->lock);
            return 0;
        }
        release(&p->lock);
    }
    return -1;
}
```

Design

- kernel/proc.c
- void sleep(void *chan, struct spinlock *lk)
 - process의 chan을 설정하고 state를 SLEEPING으로 바꿔주는 함수인데, thread를 위해 특별히 고칠 부분은 없다.

```
void
sleep(void *chan, struct spinlock *lk)
{
    struct proc *p = myproc();

    // Must acquire p->lock in order to
    // change p->state and then call sched.
    // Once we hold p->lock, we can be
    // guaranteed that we won't miss any wakeup
    // (wakeup locks p->lock),
    // so it's okay to release lk.

    acquire(&p->lock); //DOC: sleeplock1
    release(lk);

    // Go to sleep.
    p->chan = chan;
    p->state = SLEEPING;
    sched();
    // Tidy up.
    p->chan = 0;

    // Reacquire original lock.
    release(&p->lock);
    acquire(lk);
}
```

Design

- kernel/proc.c int wait(uint64 addr)
 - proc table을 돌면서 현재 process를 부모로 하는 process를 찾고 state가 ZOMBIE라면 해당 process를 free해준 뒤 process의 pid를 return한다.
 - thread를 join할 때 참고하면 될 듯하다.

```
int
wait(uint64 addr)
{
    struct proc *pp;
    int havekids, pid;
    struct proc *p = myproc();

    acquire(&wait_lock);

    for(;;){
        // Scan through table looking for exited children.
        havekids = 0;
        for(pp = proc; pp < &proc[NPROC]; pp++){
            if(pp->parent == p){
                // make sure the child isn't still in exit() or swtch().
                acquire(&pp->lock);

                havekids = 1;
                if(pp->state == ZOMBIE){
                    // Found one.
                    pid = pp->pid;
                    if(addr != 0 && copyout(p->pagetable, addr, (char *)&pp->xstate,
                                           sizeof(pp->xstate)) < 0) {
                        release(&pp->lock);
                        release(&wait_lock);
                        return -1;
                    }
                }
                freeproc(pp);
                release(&pp->lock);
                release(&wait_lock);
                return pid;
            }
        }
        release(&pp->lock);
    }
}
```


Design

- kernel/proc.c int exit(int status)
 - proccess를 종료하는 함수이다.
 - 과제 명세에 맞게 main thread가 exit()을 호출하면 main thread의 모든 thread들이 종료되고, 그냥 thread만 exit()을 호출할 경우 해당 thread만 종료될 수 있도록 수정해야 한다.

```
void
exit(int status)
{
    struct proc *p = myproc();

    if(p == initproc)
        panic("init exiting");

    // Close all open files.
    for(int fd = 0; fd < NOFILE; fd++){
        if(p->ofile[fd]){
            struct file *f = p->ofile[fd];
            fileclose(f);
            p->ofile[fd] = 0;
        }
    }

    begin_op();
    iput(p->cwd);
    end_op();
    p->cwd = 0;

    acquire(&wait_lock);

    // Give any children to init.
    reparent(p);

    // Parent might be sleeping in wait().
    wakeup(p->parent);

    acquire(&p->lock);

    p->xstate = status;
    p->state = ZOMBIE;

    release(&wait_lock);

    // Jump into the scheduler, never to return.
    sched();
    panic("zombie exit");
}
```

Design

- kernel/proc.c
- int clone(void(*fcn)(void*, void*), void *arg1, void *arg2, void *stack)
 - 새로운 thread를 생성하는 함수이다. process를 만드는 fork() 함수를 참고하여 만들 계획이다.
 - fork()와의 차이점은 다음과 같다.
 - 1. thread는 pagetable을 공유한다.
 - 2. thread의 trapframe 위치는 main thread와 같은 곳에 mapping되면 안되고, 같은 main thread 하의 다른 thread들도 서로 다른 위치에 trapframe이 mapping되어야 한다.
 - 3. fork()에서 자식 process는 부모 process의 trapframe을 복사하지만, thread의 경우 trapframe의 약간의 수정이 필요하다. 자세한 건 구현에서 설명하겠다.
- int join(void *stack)
 - child thread가 종료될 때까지 기다리는 함수이다. child process가 종료될 때까지 기다리는 wait() 함수를 참고하여 만들 계획이다.

Implementation Overview

- kernel/proc.h
 1. struct proc 수정
- kernel/proc.c
 1. clone() 추가
 2. allocthread() 추가
 3. alloctid() 추가
 4. thread_pagetable() 추가
 5. join 추가
 6. freethread() 추가
 7. procinit() 수정
 8. allocproc() 수정
 9. freeproc() 수정
 10. growproc() 수정
 11. exit() 수정
 12. kill() 수정
- kernel/exec.c
 1. exec() 수정
- user/thread.c
 1. thread_create() 추가
 2. thread_join() 추가

Implementation Overview

- kernel/proc.h

1. struct proc 수정

proc 구조체가 thread도 지원하도록 member 변수들을 추가했다.

int is_thread:

thread인 경우 1, main thread이거나 process의 경우 0

int tid:

thread의 독립적인 id를 설정.

void *ustack:

clone()에서 thread의 user stack 위치를 저장하여 join()에서 해당 stack을 free()하기 위해 필요.

```
struct proc {
    struct spinlock lock;

    // p->lock must be held when using these:
    enum procstate state;      // Process state
    void *chan;                // If non-zero, sleeping on chan
    int killed;                // If non-zero, have been killed
    int xstate;                // Exit status to be returned to parent's wait
    int pid;                   // Process ID

    int is_thread;
    int tid;
    void *ustack;
```

Implementation Overview

- kernel/proc.c

1. clone() 추가

모든 thread의 parent pointer로 main thread를 가리킨다.

새로운 thread를 생성할 때 main thread와 pagetable, pid, sz, 등 정보를 공유해야 하기 때문에 myproc()이 thread인 경우 `p = cur->parent;`를 통해 main thread를 가리킬 수 있도록 한다.

thread의 부모를 main thread로 설정하고 main thread와 pid를 공유한다.

`allocthread()`로 새로운 thread를 생성한다. `allocthread()` 설명은 뒤에 하겠다.

thread의 trapframe이 main thread의 trapframe과 겹치지 않도록 main thread의 trapframe보다 `tid * PGSIZE`만큼 아래에 thread의 trapframe을 mapping해준다. 그러기 위해 `trapframe_va`를 수정하고 새로 만든 `thread_pagetable()`에서 mapping을 진행한다. 이에 대한 설명도 뒤에 하겠다. `trapframe_va`는 기존 xv6에서는 없는 member 변수지만, 이번 과제를 위해 임의로 추가된 것이다.

`pagetable`을 생성하는 것을 실패 할 경우 `freethread()`를 통해 생성 전으로 복구한다.

```
int
clone(void(*fcn)(void*, void*), void *arg1, void *arg2, void *stack)
{
    int i, pid;
    struct proc *np;
    struct proc *p;
    struct proc *cur = myproc();

    if((uint64)stack == 0 || (uint64)stack % PGSIZE != 0){
        return -1;
    }

    if(cur->is_thread){
        p = cur->parent;
    }else{
        p = cur;
    }

    // Allocate thread.
    if((np = allocthread()) == 0){
        return -1;
    }

    np->parent = p;
    np->pid = p->pid;
    np->trapframe_va = p->trapframe_va - np->tid * PGSIZE;
    np->pagetable = p->pagetable;
    if(thread_pagetable(np) < 0){
        printf("mappage error\n");
    }
    if(np->pagetable == 0){
        freethread(np);
        release(&np->lock);
        return -1;
    }

    np->sz = p->sz;
```

Implementation Overview

- kernel/proc.c

1. clone() 추가

thread의 trapframe을 수정해 정상적으로 thread가 실행될 수 있도록 한다.

trapframe->epc = (uint64)fcn;

- thread가 시작할 위치를 정한다.

trapframe->sp = (uint64)stack + PGSIZE;

- thread의 userstack에서 PGSIZE 만큼 위를 stack pointer로 정한다.

trapframe->a0 = (uint64)arg1;

- thread의 argument1, argument2를 각각 a0, a1에 저장한다.

np->ustack = stack;

- thread의 userstack 시작 주소를 ustack에 저장해 나중에 join()에서 userstack 주소를 찾아 free할 수 있도록 한다.

```
np->trapframe->epc = (uint64)fcn;
np->trapframe->sp = (uint64)stack + PGSIZE;

// Cause fork to return 0 in the child.
np->trapframe->a0 = (uint64)arg1;
np->trapframe->a1 = (uint64)arg2;

// increment reference counts on open file descriptors.
for(i = 0; i < NOFILE; i++)
    if(p->ofile[i])
        np->ofile[i] = filedup(p->ofile[i]);
if(p->cwd)
    np->cwd = idup(p->cwd);
else
    np->cwd = 0;

safestrcpy(np->name, p->name, sizeof(p->name));

pid = np->pid;
np->ustack = stack;

release(&np->lock);

acquire(&np->lock);
np->state = RUNNABLE;
release(&np->lock);

return pid;
}
```

Implementation Overview

- kernel/proc.c

- 2. allocthread() 추가

allocproc()과 구분하기 위해 thread를 할당하는 allocthread()를 만들었다.

allocproc()과 큰 차이는 없지만,

- 1. process의 is_thread를 1로 설정해 thread임을 명시한다.

- 2. main thread와 pagetable을 공유하기 때문에 allocproc()에서 pagetable을 생성하는 proc_pagetable()을 삭제했다.

```
static struct proc*
allocthread(void)
{
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if(p->state == UNUSED) {
            goto found;
        } else {
            release(&p->lock);
        }
    }

    return 0;

found:
    p->tid = allockid();
    p->state = USED;
    p->is_thread = 1;

    // Allocate a trapframe page.
    if((p->trapframe = (struct trapframe *)kalloc()) == 0){
        freethread(p);
        release(&p->lock);
        return 0;
    }

    // Set up new context to start executing at forkret,
    // which returns to user space.
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)forkret;
    p->context.sp = p->kstack + PGSIZE;

    return p;
}
```

Implementation Overview

- kernel/proc.c

- 3. allocpid() 추가

- 6. procinit() 수정

thread는 main thread와 pid를 공유하게 설계했다.

하지만 thread마다 구분을 하기 위해 thread id인 tid를 보유하게 했다.

pid가 다르다면 tid가 같아도 문제가 발생하지 않지만,

간단한 설계를 위해 모든 thread의 tid가 다르게 설정했다.

allocpid()와 동일하게 구현했으며 race condition을 방지하기 위해 tid_lock을 추가했다.

```
int nextpid = 1;
int nexttid = 1;
struct spinlock pid_lock;
struct spinlock tid_lock;
```

```
void
procinit(void)
{
    struct proc *p;

    initlock(&pid_lock, "nextpid");
    initlock(&tid_lock, "nexttid");
    initlock(&wait_lock, "wait_lock");
    for(p = proc; p < &proc[NPROC]; p++) {
        initlock(&p->lock, "proc");
        p->state = UNUSED;
        p->kstack = KSTACK((int) (p - proc));
    }
}
```

```
int
allocpid()
{
    int tid;

    acquire(&tid_lock);
    tid = nexttid;
    nexttid = nexttid + 1;
    release(&tid_lock);

    return tid;
}
```


Implementation Overview

- kernel/proc.c

- 4. thread_pagetable() 추가

proc_pagetable()을 참고해서 만들었다.

thread의 경우 pagetable을 main thread와 공유하기 때문에 proc_pagetable()을 한 번 더 실행 할 필요는 없다.

하지만 thread를 위한 trapframe을 mapping해야 하기 때문에, mappage()를 사용해 thread의 trapframe을 trapframe_va 위치에 mapping해준다.

```
int
thread_pagetable(struct proc* p){
    if(p->is_thread){
        if(mappages(p->pagetable, p->trapframe_va, PGSIZE,
                    (uint64)(p->trapframe), PTE_R | PTE_W) < 0){
            return -1;
        }
    }
    return 0;
}
```

Implementation Overview

- kernel/proc.c

- 5. join 추가

child process가 종료되는 것을 기다리는 wait()을 참고했다.

is_thread가 1이면서 myproc()의 pid와 같은 pid를 갖고 있으면, 현재 process의 thread임을 나타내기 때문에, 해당 thread가 ZOMBIE일 경우 freethread()를 통해 회수했다.

또한 user mode에서 malloc 했던 user stack을 free해주기 위해 stack 인자에 ustack을 전달한다.

```
int
join(void **stack)
{
    struct proc *pp;
    int havekids, pid;
    struct proc *p = myproc();

    acquire(&wait_lock);

    for(;;){
        // Scan through table looking for exited children.
        havekids = 0;
        for(pp = proc; pp < &proc[NPROC]; pp++){
            if(pp->is_thread && pp->pid == p->pid){
                // make sure the child isn't still in exit() or swtch().
                havekids = 1;
                acquire(&pp->lock);
                if(pp->state == ZOMBIE){
                    // Found one.
                    pid = pp->pid;
                    *stack = pp->ustack;
                    freethread(pp);
                    release(&pp->lock);
                    release(&wait_lock);
                    return pid;
                }
                release(&pp->lock);
            }
        }

        // No point waiting if we don't have any children.
        if(!havekids || killed(p)){
            release(&wait_lock);
            return -1;
        }

        // Wait for a child to exit.
        sleep(p, &wait_lock); //DOC: wait-sleep
    }
}
```

Implementation Overview

- kernel/proc.c

- 6. freethread() 추가

freeproc()을 참고해 만들었다.

freeproc()에서는 proc_freepagetable()을 통해 pagetable에서 TRAMPOLINE과 TRAPFRAME을 uvmunmap()으로 mapping을 해제해준 뒤 해당 pagetable을 uvmfree하는데,

thread는 main thread와 pagetable을 공유하기 때문에 함부로 pagetable을 uvmfree하면 안되고, TRAMPOLINE과 TRAPFRAME도 uvmunmap하면 안된다. 따라서 해당 부분은 제거했다.

하지만 thread의 trapframe은 trapframe_va에 저장된 위치에 mapping되어 있으므로, uvmunmap()을 통해 mapping을 해제시켜줘야 한다.

```
void
proc_freepagetable(pagetable_t pagetable, uint64 sz)
{
    uvmunmap(pagetable, TRAMPOLINE, 1, 0);
    uvmunmap(pagetable, TRAPFRAME, 1, 0);
    uvmfree(pagetable, sz);
}
```

```
void
freethread(struct proc *p)
{
    if(p->is_thread && p->trapframe)
        kfree((void*)p->trapframe);
    p->trapframe = 0;
    if(p->is_thread && p->pagetable && p->trapframe_va)
        uvmunmap(p->pagetable, p->trapframe_va, 1, 0);
    p->trapframe_va = 0;
    p->sz = 0;
    p->pid = 0;
    p->parent = 0;
    p->name[0] = 0;
    p->chan = 0;
    p->killed = 0;
    p->xstate = 0;
    p->state = UNUSED;
    p->is_thread = 0;
    p->tid = 0;
    p->ustack = 0;
}
```

Implementation Overview

- kernel/proc.c

- 7. allocproc() 수정

thread를 할당하는 것이 아닌, process를 할당하는 것이므로 해당 process의 is_thread member 변수를 0으로 설정한다.

```
static struct proc*
allocproc(void)
{
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if(p->state == UNUSED) {
            goto found;
        } else {
            release(&p->lock);
        }
    }

    found:
    p->pid = allocpid();
    p->state = USED;
    p->is_thread = 0;

    // Allocate a trapframe page.
    if((p->trapframe = (struct trapframe *)kalloc()) == 0){
        freeproc(p);
        release(&p->lock);
        return 0;
    }

    // An empty user page table.
    p->pagetable = proc_pagetable(p);
    if(p->pagetable == 0){
        freeproc(p);
        release(&p->lock);
        return 0;
    }

    // Set up new context to start executing at forkret,
    // which returns to user space.
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)forkret;
    p->context.sp = p->kstack + PGSIZE;

    return p;
}
```

Implementation Overview

- kernel/proc.c
 - 8. freeproc() 수정
proc.h에서 추가한 member 변수를 0으로 초기화한다.

```
static void
freeproc(struct proc *p)
{
    if(p->trapframe)
        kfree((void*)p->trapframe);
    p->trapframe = 0;
    p->trapframe_va = 0;
    if(p->pagetable)
        proc_freepagetable(p->pagetable, p->sz);
    p->pagetable = 0;
    p->sz = 0;
    p->pid = 0;
    p->parent = 0;
    p->name[0] = 0;
    p->chan = 0;
    p->killed = 0;
    p->xstate = 0;
    p->state = UNUSED;
    p->is_thread = 0;
    p->tid = 0;
    p->ustack = 0;
}
```

Implementation Overview

- kernel/proc.c

- 9. growproc() 수정

myproc()이 현재 thread일 경우 myproc()의 parent는 main thread를 가리키기 때문에 main thread로 이동해서 uvmalloc() 또는 uvmdealloc()을 진행한 뒤, main thread와 현재 thread의 sz를 변경해준다.

```
int
growproc(int n)
{
    uint64 sz;
    struct proc *p;
    struct proc *cur = myproc();

    if(cur->is_thread){
        p = cur->parent;
    } else{
        p = cur;
    }

    sz = p->sz;
    if(n > 0){
        if((sz = uvmalloc(p->pagetable, sz, sz + n, PTE_W)) == 0) {
            return -1;
        }
    } else if(n < 0){
        sz = uvmdealloc(p->pagetable, sz, sz + n);
    }

    p->sz = sz;
    cur->sz = p->sz;
    return 0;
}
```

Implementation Overview

- kernel/proc.c

- 10. exit() 수정

main thread는 is_thread가 0이기 때문에 main thread가 exit()을 호출하면 is_thread가 1이고 UNUSED가 아니며 main thread와 pid를 동일하게 갖는 thread를 freethread()를 통해 회수할 수 있게 수정한다.

```
void
exit(int status)
{
    struct proc *p = myproc();

    if(p == initproc)
        panic("init exiting");

    // If this is the main thread, kill all threads sharing the same pagetable.
    if(!p->is_thread){
        struct proc *pp;
        for(pp = proc; pp < &proc[NPROC]; pp++) {
            if(pp->is_thread && pp->state != UNUSED && pp->pid == p->pid){
                freethread(pp);
            }
        }
    }

    // Close all open files.
    for(int fd = 0; fd < NOFILE; fd++){
        if(p->ofile[fd]){
            struct file *f = p->ofile[fd];
            fileclose(f);
            p->ofile[fd] = 0;
        }
    }
}
```

Implementation Overview

- kernel/proc.c

- 11. kill() 수정

한 thread가 kill syscall을 호출하면 process 내의 모든 thread가 종료되어야 한다. 하지만 기존 kill()은 proc table을 돌면서 pid를 갖는 process를 찾으면 killed를 1로 바꾸고 return 0;를 호출해 끝나버린다.

따라서 proc table을 돌면서 pid를 갖는 모든 process나 thread의 killed를 1로 바꿔서 나중에 종료될 수 있도록 한다.

found 변수를 추가해 pid를 갖는 process나 thread가 없을 경우 -1을 return할 수 있도록 한다.

```
int
kill(int pid)
{
    struct proc *p;
    int found = 0;

    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->pid == pid){
            found = 1;
            p->killed = 1;
            if(p->state == SLEEPING){
                // Wake process from sleep().
                p->state = RUNNABLE;
            }
        }
        release(&p->lock);
    }
    if(found) return 0;
    return -1;
}
```


Implementation Overview

- kernel/exec.c

1. exec() 수정

thread에서 exec syscall을 호출할 경우, 현재 process의 thread는 모두 종료되고 새로운 process가 실행되어야 한다.

원래 exec()은 fork() 처럼 새로운 process를 생성하는 것이 아니라 현재 process를 새로운 process로 교체하는 함수이다. 즉 thread에서 exec()을 호출할 경우 그 thread가 새로운 process가 되어야 하는 것이다.

따라서 cpu가 잡고 있는 thread를 main thread로 변경해주고 그 외의 thread는 freethread()로 회수해줬다. thread의 parent도 원래 main thread의 parent로 설정하고 다른 thread의 parent를 현재 thread로 가리키게 만든다.

이후 현재 thread의 trapframe mapping을 uvmunmap()으로 회수하고 is_thread를 0으로 trapframe_va를 TRAPFRAME으로 바꾸면 현재 thread는 process와 동일한 상태가 된다. 이후 기존 exec()를 진행하면 정상적으로 exec()이 작동한다.

```
int
exec(char *path, char **argv)
{
    char *s, *last;
    int i, off;
    uint64 argc, sz = 0, sp, ustack[MAXARG], stackbase;
    struct elfhdr elf;
    struct inode *ip;
    struct proghdr ph;
    pagetable_t pagetable = 0, oldpagetable;
    struct proc *p = myproc();

    if(p->is_thread){
        p->parent = p->parent->parent;
        for (struct proc *pp = proc; pp < &proc[NPROC]; pp++) {
            if (pp != p && pp->pid == p->pid) {
                acquire(&pp->lock);
                if(pp->is_thread) freethread(pp);
                else pp->is_thread = 1;
                if (pp->state != UNUSED) {
                    pp->killed = 1;
                    if (pp->state == SLEEPING)
                        pp->state = RUNNABLE;
                }
                pp->parent = p;
                pp->trapframe_va = 0;
                release(&pp->lock);
            }
        }
        uvmunmap(p->pagetable, p->trapframe_va, 1, 0);
        p->is_thread = 0;
        p->trapframe_va = TRAPFRAME;
    }

    begin_op();
}
```

Implementation Overview

- user/thread.c

1. thread_create() 추가

새로운 thread를 만들 때 user stack의 주소가 PGSIZE로 정렬되어야 하기 때문에 이를 보장하기 위해 2 * PGSIZE만큼 malloc을 한 뒤 PGSIZE로 정렬해주었다.

이후 clone() syscall을 호출한다.

- user/thread.c

2. thread_join() 추가

thread가 종료될 때 join() syscall을 통해 user stack의 주소를 받아와서 free()해준다.

```
int
thread_create(void (*start_routine)(void *, void *), void *arg1, void *arg2)
{
    void *base = malloc(2 * PGSIZE);
    if(base == 0)
        return -1;

    uint64 addr = (uint64)base;
    void *stack = (void *)((addr % PGSIZE == 0) ? addr : addr + (PGSIZE - addr % PGSIZE));
    int pid = clone(start_routine, arg1, arg2, stack);
    if(pid < 0){
        free(base);
        return -1;
    }

    return pid;
}
```

```
int
thread_join()
{
    void *stack;
    int pid = join((void**)&stack);
    if(pid < 0)
        return -1;
    free(stack);
    return pid;
}
```

Implementation

- System call 구현

kernel/syscall.h

```
#define SYS_clone 22
#define SYS_join 23
```

kernel/syscall.c

```
extern uint64 sys_clone(void);
extern uint64 sys_join(void);
```

```
[SYS_clone] sys_clone,
[SYS_join] sys_join
```

user/usys.pl

```
entry("clone");
entry("join");
```

user/user.h

```
int clone(void(*fcn)(void*, void*), void *arg1, void *arg2, void *stack);
int join(void **stack);
```

kernel/sysproc.c

```
uint64
sys_clone(void)
{
    uint64 fcn, arg1, arg2, stack;

    argaddr(0, &fcn);
    argaddr(1, &arg1);
    argaddr(2, &arg2);
    argaddr(3, &stack);

    return clone((void (*)(void*, void*))fcn, (void*)arg1, (void*)arg2, (void*)stack);
}
```

```
uint64
sys_join(void)
{
    uint64 ustack;
    argaddr(0, &ustack);

    void *stack = 0;
    int pid = join(&stack);
    if(pid < 0)
        return -1;

    if (copyout(myproc()->pagetable, ustack, (char *)&stack, sizeof(void *)) < 0)
        return -1;

    return pid;
}
```

유저 프로그램에서 thread_join()을 호출했을 때, kernel 공간에서 종료된 thread의 user stack 주소를 user space에 넘겨주기 위해 copyout()을 사용한다.

join()에서 stack 주소는 kernel에 있지만 (void *stack) 유저 프로그램이 그 값을 알아야 한다(void **ustack).

kernel은 user address space에 직접 접근할 수 없기 때문에 copyout()을 통해 user space 포인터로 stack 주소를 복사해야 유저가 해제 가능

Results

- 추가로 주어진 테스트 파일 thread_test.c 실행 결과다.

```
xv6 kernel is booting

init: starting sh
$ thread_test

[TEST#1]
Thread 0 start
Thread 1 start
Thread 1 end
Thread 2 start
Thread 2 end
Thread 3 start
Thread 3 end
Thread 4 start
Thread 4 end
Thread 0 end
TEST#1 Passed

[TEST#2]
Thread 0 start, iter=0
Thread 0 end
Thread 1 start, iter=1000
Thread 1 end
Thread 2 start, iter=2000
Thread 2 end
Thread 3 start, iter=3000
Thread 3 end
Thread 4 start, iter=4000
Thread 4 end
TEST#2 Passed
```

```
[TEST#3]
Thread 0 start
Thread 1 start
Thread 2 start
Thread 3 start
Thread 4 start
Child of thread 0 start
Child of thread 1 start
Child of thread 2 start
Child of thread 3 start
Child of thread 4 start
Child of thread 0 end
Child of thread 1 end
Child of thread 2 end
Child of thread 3 end
Child of thread 4 end
Thread 0 end
Thread 1 end
Thread 2 end
Thread 3 end
Thread 4 end
TEST#3 Passed

[TEST#4]
Thread 0 sbrk: old break = 0x0000000000025000
Thread 0 sbrk: increased break by 14000
new break = 0x0000000000049010
Thread 1 size = 0x0000000000049010
Thread 2 size = 0x0000000000049010
Thread 3 size = 0x0000000000049010
Thread 4 size = 0x0000000000049010
Thread 0 sbrk: free memory
Thread 0 end
Thread 1 end
Thread 2 end
Thread 3 end
Thread 4 end
TEST#4 Passed
```

```
[TEST#5]
Thread 0 start, pid 9
Thread 1 start, pid 9
Thread 2 start, pid 9
Thread 3 start, pid 9
Thread 4 start, pid 9
TEST#5 Passed
```

```
[TEST#6]
Thread 0 start
Thread 1 start
Thread 2 start
Thread 3 start
Thread 4 start
Executing...
Thread exec test 0
TEST#6 Passed
```

All tests passed. Great job!!

Troubleshooting

- thread를 처음 구현할 때 trapframe을 단순히 참조 연산자 *를 사용해 값을 복사해주면 된다고 생각했다. 그래서 이로 인한 문제가 발생했을 때 문제점을 파악하지 못해 많은 시간을 보냈다. lab ppt를 분석했을 때 thread는 같은 pagetable을 공유하기 때문에 trapframe의 mapping 위치가 중요하다는 것을 깨달았고, trapframe이 겹치지 않도록 조절해주었다.
- exec()을 수정 할 때, panic: leaf가 발생하며 회수 되지 않은 부분이 있음을 알았다. 이것 역시 thread가 pagetable을 공유해서 발생한 문제였다. 기존 TRAPFRAME과 다른 위치에 mapping한 trapframe을 회수하고 현재 thread가 main thread가 될 수 있도록 회수 타이밍을 알맞게 수정했다.
- thread 종료 후 thread_join()을 호출했을 때, stack 주소를 제대로 전달받지 못해 panic이 발생했다. sys_join()에서 thread의 kernel stack 주소를 user space로 잘 못 넘겨서 발생한 문제였다. kernel이 user address space에 직접 접근할 수 없다는 사실을 알고 copyout()을 사용해 stack 주소를 user address space로 제대로 전달할 수 있었다.